App Usage Assumptions & Estimations

About the Product & Use Cases

- The application is a carbon footprint registry and certification system.
- The STAGING environment supports the same core application features as a future PROD environment, but is deployed with minimal, cost-optimized resources and reduced capacity.
- Used for:
  - Internal QA and functional testing
  - External stakeholder demos
  - Pilot tests with real client onboarding
- Workflow includes:
  - Document upload
  - Report generation
  - Status tracking
  - User notifications
- No external business systems integrations; infrastructure dependencies (auth, APM) remain active
- No batch operations, except:
  - Occasional internal maintenance
  - Manual data cleanup or resets
- Although STAGING is deployed with minimal resources, any pilot involving real organizations and real data must meet baseline production-grade requirements for data protection, security, recovery, and operational control.

About Traffic & Load

- **10–30 Daily Active Users (DAU)**.
- Typical users:
  - Internal team members
  - Selected external testers
  - Some country's organizations
- **100–300 API requests/day per user**.
- Estimated traffic:
  - ~2,000–9,000 API requests/day
  - ~60,000–270,000 API requests/month
- Peak traffic:
  - **1–3 requests per second (RPS)** during demos or test sessions.
- Traffic pattern:
  - Intermittent
  - Mostly during work hours (8–10 hours/day)
- No sustained load or concurrency expected.
- No bulk or stress-test traffic unless explicitly triggered.
- Backend is stateless.

About the API and Containers

- Same API codebase as PROD.
- Execution time:
  - ~2 seconds per request (worst-case, acceptable for STAGING).
- Low concurrency expected.
- No scaling-up triggers needed.

- Designed to:
  - Scale minimally
  - Favor cost efficiency over performance headroom
- No strict SLA requirements.

About the Database

- Same schema as PROD:
  - ~45 tables
  - All tables with foreign keys
- No PostGIS.
- Queries are simple (CRUD, joins, no heavy analytics).
- Data characteristics:
  - Limited test and demo data
  - Periodic resets or truncation possible
- Database growth:
  - Small and controlled
  - Expected DB size: 5-20 GB
- Baseline IOPS sufficient for <10 concurrent queries
- Backup and restore used mainly for:
  - Environment recovery
  - Demo preparation

About the Storage

- Stores:
  - Test documents
  - Images

- Generated PDF reports
- Metadata

- Uploads are:
  - User-driven
  - Low volume
  - Often repeated or disposable
- Expected storage size:
  - Target: <50 GB; automated alerts at 40 GB; manual purge every 90 days
- No long-term retention requirements.
- Data can be periodically purged.

About Logs & Observability

- Application Insights enabled.
- Log ingestion:
  - Reduced verbosity compared to PROD
  - Sampling enabled (≥10%, adjustable).
- Retention:
  - **30 days** (sufficient for debugging and validation).
- Focus:
  - Error tracking
  - Basic performance insights
  - Demo issue diagnosis

About Authentication

- Same authentication mechanism as PROD (Entra ID).
- Expected users:

    - **20–50 total users**
- No real end-user onboarding.
- Identity data may be reused or mocked.
- No growth expectations.

About Email / Notifications

- Emails used for:
    - Functional testing
    - Demo flows
- Very low volume.
- Average email size:
    - Up to **5 MB** (same as PROD, but infrequent).
- No high-throughput or reliability requirements.
- Can use:
    - Sandbox email provider
    - Restricted distribution lists

About Reliability & Failure Tolerance (STAGING)

The STAGING environment targets **baseline reliability appropriate for testing, demos, and pilot activities**, while remaining **cost-optimized and operationally simple**.

Because STAGING may handle **real organizational data during pilots**, it must still meet **minimum production-grade safeguards**, even though it does not provide production-level availability guarantees.

- **Acceptable error rate:** ≤ **5%** of requests (server-side errors)
- **API timeout threshold: 10 seconds** (hard timeout)

- **Recovery Time Objective (RTO):** ≤ **24 hours**
- **Recovery Point Objective (RPO):** ≤ **24 hours**

These targets reflect the **non-critical nature of STAGING**, while ensuring data protection, safe pilot operation, and predictable recovery.

Acceptable Error Rate Explanation

The recommendation is **≤ 5% of requests resulting in server-side errors (5xx)**.

This makes sense because:

- STAGING is used for:
  - Functional testing
  - Demos
  - Pilot validation
- Occasional errors are acceptable and expected
- Stability is preferred, but **not guaranteed**

Errors should be:

- Visible
- Diagnosable
- Recoverable without data corruption

Interpretation:

- ≥ **95% success rate** is acceptable
- Errors must **not**:
  - Corrupt data

- Lose pilot submissions
- Break demo flows irreversibly

Comparison:

System Type	Typical Error Rate
Development / QA	5–10%
STAGING (pilots & demos)	**≤ 5%**
Production regulatory systems	≤ 0.5%

✔️ **5% balances realism and cost efficiency**

Timeout Threshold Explanation

The recommendation is **10 seconds hard timeout for API requests**.

This matches PROD intentionally because:

- Code paths are the same
- Timeout behavior must be validated before production
- Differences in behavior between environments would hide issues

Practical breakdown:

- Normal requests: 1–3 seconds
- Slow requests: 3–6 seconds
- Hard cutoff: 10 seconds

Requests exceeding this should:

- Fail cleanly
- Be retried during testing

- Surface performance issues early

Comparison:

System Type	Timeout
Local development	Variable
STAGING	**10s**
Production	10s

✔️ **Keeping the same timeout as PROD is intentional and correct**

RTO – Recovery Time Objective Explanation

The recommendation is **≤ 24 hours**.

This means that if STAGING goes down:

- The system must be restored **within one business day**

This is appropriate because:

- No contractual SLA applies
- No continuous user dependency exists
- Downtime can be communicated and planned around

Comparison:

Platform Type	RTO
Production regulatory systems	≤ 4 hours
STAGING	**≤ 24 hours**
Internal dev environments	Best effort

✔️ **24 hours avoids overengineering while maintaining discipline**

RPO – Recovery Point Objective Explanation

The recommendation is **≤ 24 hours**.

This means:

- Maximum acceptable data loss: **up to one day**
- Recent pilot data loss is acceptable if clearly communicated

This is acceptable because:

- Data can often be re-created
- Pilots are exploratory by nature
- No legal or regulatory commitments depend on STAGING data

Comparison:

System Type	RPO
Production regulatory systems	≤ 15 minutes
STAGING	**≤ 24 hours**
Development	Not defined

✔️ **24 hours is pragmatic and cost-aware**

Sanity Check

Are these targets too relaxed? **No.**

These targets:

- **Do not require**:
  - High availability
  - Multi-region setups

    - On-call rotation
- **Do require**:
    - Automated backups
    - Restore procedures
    - Basic monitoring and alerting

This creates a **safe but economical STAGING environment**, suitable for pilots with real organizations without incurring production-grade costs.

About AI Usage

The STAGING environment includes **AI-powered features** used to validate functionality, user experience, and system integration prior to production rollout.

AI capabilities are **not intended for high-volume or continuous usage** in STAGING and are limited to **functional testing, demos, and pilot scenarios**.

AI features in STAGING could be used for:

- AI-assisted reading and understanding of uploaded documents
- Retrieval-Augmented Generation (RAG) workflows using indexed documents
- Summarization and analysis of reports and evidence files
- End-to-end demo flows involving AI-generated outputs

AI is **user-triggered only**.

No background, batch, or automated AI processing is performed.

Estimated AI usage and traffic:

- **~1,000 AI requests per month**
- Usage is intermittent and concentrated during:
    - Internal QA sessions
    - External demos

    - Pilot testing with selected users
- No sustained or concurrent AI load is expected.
- Following a conservative sizing and cost estimation, the following **upper-bound assumptions** are used (per request):
    - **New input (non-cached):** ~5,000 tokens
    - **Cached / repetitive input:** ~3,000 tokens
    - **AI output:** ~1,500 tokens (correspond to approximately 1,100–1,200 words, equivalent to 2–3 pages of a Word document, representing a long-form AI response)

These assumptions intentionally overestimate typical usage to provide safety margin.

The following are **not part of STAGING AI usage (out of scope)**:

- High-volume conversational AI
- Large-scale document ingestion
- Batch or scheduled AI processing
- SLA-driven or latency-critical AI workloads
- Public or anonymous AI access

The AI configuration must provides **production-like AI behavior** while maintaining **strict cost efficiency and operational simplicity**, ensuring the STAGING environment remains predictable and safe.

We suggest to use Azure Functions in STAGING for:

- Orchestrating **LangChain workflows**
- Coordinating:
    - LLM calls

    - Retrieval (RAG) steps
    - Prompt assembly
    - Response post-processing
- Triggered **only by user actions**
- No background, batch, or scheduled AI jobs
- No public or anonymous access

All heavy computation (LLM inference, embeddings) is executed by **external managed AI services**, not inside the Function runtime.

Although the absolute number of AI requests in STAGING and early PROD may appear similar, the usage patterns differ significantly. STAGING AI usage is concentrated during demos, QA, and pilot scenarios with repeated flows, while PROD AI usage starts conservatively and grows gradually as adoption increases across organizations. For this reason, initial PROD sizing assumes modest AI usage, with clear headroom for future expansion.

About the Background Tasks

The analysis identifies three categories of background tasks needed for the app:

1️⃣ **Database Cleanup**

Examples include:

- Deleting obsolete or expired records
- Clearing orphaned data
- Running lightweight maintenance logic

Typical execution time: 20–60 seconds

Frequency assumption: 1 execution per hour

2️⃣ **Reminders & Pending Tasks**

Used to:

- Check for pending workflows
- Send notifications (email, webhook, queue messages)
- Trigger periodic application-level checks

Typical execution time: 1–10 seconds

Frequency assumption: Every 10 minutes

3️⃣ Data Lifecycle Management

Examples include:

- Archiving old documents
- Moving data between storage tiers
- Expiring or reclassifying items

Typical execution time: 5–30 seconds

Frequency assumption: Every 2 hours

📊 Estimated Monthly Executions

Based on the execution patterns:

Task Type	Frequency	Estimated Monthly Executions
Database Cleanup	1/hour	~720/month
Reminders	1/10 minutes	~4,300/month
Data Lifecycle	1/2 hours	~360/month

Total estimated executions: ~5,400 executions/month

⏱️ Execution Duration Assumptions

Typical durations are:

Task Type	Duration
Database Cleanup	~40 seconds

Task Type	Duration
Reminders	~5 seconds
Data Lifecycle	~15 seconds

Average memory: **512 MB (0.5 GB)**

These durations fall well within serverless best practices and do not require premium or dedicated hosting.

Key Differences vs PROD (Summary)

- Much lower traffic and concurrency
- Smaller database and storage footprint
- Shorter log retention
- Cost-optimized scaling
- No real client usage or SLA
- Data is disposable and resettable